COMPREHENSIVE PROJECT MANUAL

STUDENT INTELLIGENT CAREER RECOMMENDATION SYSTEM

PREPARED FOR:

GWANI ABDULLAHI JERE

PART I: INTRODUCTION TO THE MANUAL

1.1 Purpose and Scope of This Manual

This comprehensive manual serves as the definitive guide for understanding, maintaining, extending, and documenting the Student Intelligent Career Recommendation System. Whether you are continuing this research, adapting it for institutional deployment, or building upon it for academic purposes, this manual provides the essential knowledge and practical guidance needed to work effectively with both the technical implementation and academic documentation.

> **The manual is structured to address two primary audiences:**
>
> - **Technical Developers/Implementers**: who need to understand, modify, or extend the codebase
> - **Academic Researchers/Students**: who need to write, format, and defend the project documentation

This manual assumes basic familiarity with Python programming, machine learning concepts, and academic writing conventions. It does not assume expertise in any specific domain, making it accessible to students, educators, and administrators alike.

1.2 Project Overview

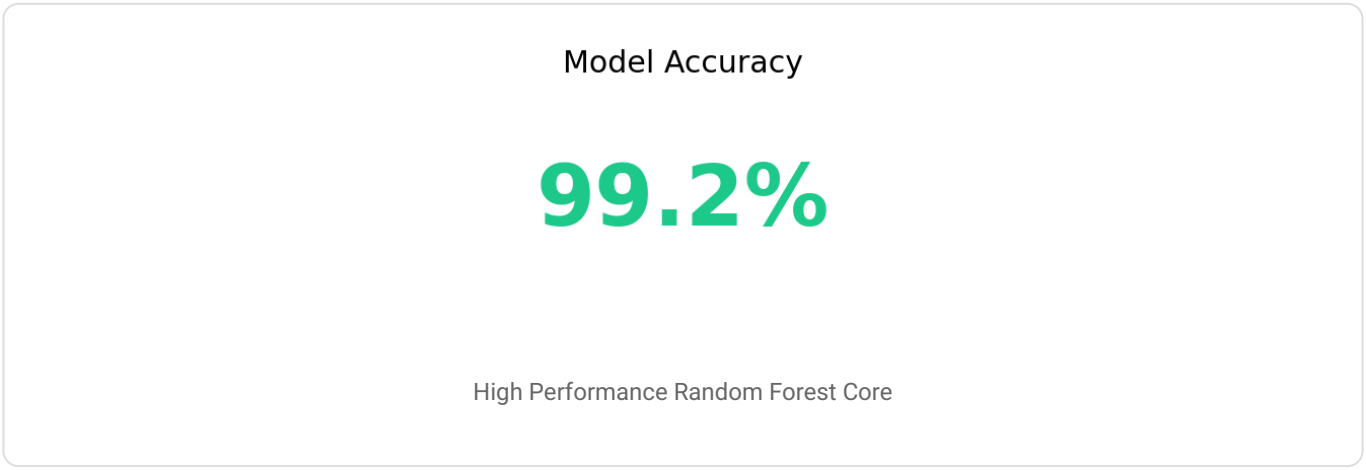
The Student Intelligent Career Recommendation System is a machine learning-based application that provides personalized career suggestions to university students based on their interests and aptitudes. The system employs a Random Forest classification algorithm trained on a dataset of 3,535 student profiles with 59 interest indicators across seven domains, mapping these to 35 distinct career paths commonly pursued in Nigerian higher education.

Key Features:

- Interactive dashboard for interest selection
- Real-time personalized recommendations
- Explainable AI features (influence factors)
- Comprehensive visual analytics
- Cloud-based deployment (Google Colab)

Technical Stack:

Python 3.12 Scikit-learn 1.3.0
Pandas 2.1.4 Numpy 1.24.0
Matplotlib 3.7.2 Seaborn 0.12.2



1.3 How to Use This Manual

This manual is designed for both linear reading and targeted reference. For new project owners:

1. First, read Sections 2.1-2.3 to understand the complete project workflow
2. Review the code structure in Section 3 before attempting modifications
3. Follow the writing guidelines in Part II when documenting your work
4. Use the troubleshooting guide in Section 4 when encountering problems

Keep this manual as a living document—update it as you make changes to the code or documentation to maintain its relevance for future project owners.

PART II: TECHNICAL IMPLEMENTATION GUIDE

2. PROJECT SETUP AND EXECUTION

2.1 Development Environment Setup

2.1.1 Google Colab Configuration

The project is designed for Google Colab, providing cloud-based computation and easy sharing. To set up:

- **Access Google Colab:** Visit colab.research.google.com
- **Create New Notebook:** Click "File" → "New Notebook"
- **Configure Runtime:** Click "Runtime" → "Change runtime type" → Select "GPU"
- **Upload Files:** Upload the dataset file (stud.csv) to the Colab environment

2.1.2 Local Environment Alternative

PYTHON CODE

```
# 1. Install Python 3.12 or higher
# 2. Create virtual environment
python -m venv career_env

# 3. Activate virtual environment
career_env\Scripts ctivate       # Windows
source career_env/bin/activate    # Mac/Linux

# 4. Install required packages
pip install pandas==2.1.4 numpy==1.24.0 scikit-learn==1.3.0
pip install matplotlib==3.7.2 seaborn==0.12.2
pip install ipywidgets==8.0.6 jupyter==1.0.0 joblib==1.3.0
```

2.2 Data Preparation

2.2.1 Dataset Requirements

The system expects a CSV file (UTF-8) with:

- **Rows:** Individual student records
- **Columns:** 59 interest features (0/1) + 1 target ('Courses')
- **Missing Values:** Minimal (< 5%)

2.2.2 Data Loading Code

PYTHON CODE

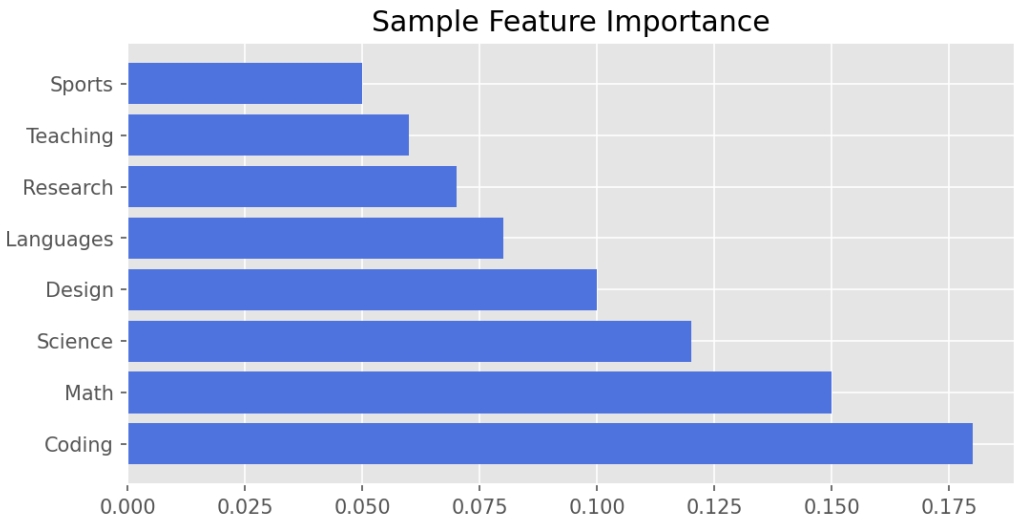
```
try:
    data_df = pd.read_csv('/content/stud.csv')
    print(f"Dataset loaded successfully! Shape: {data_df.shape}")
    print(data_df.head())
except FileNotFoundError:
    print("ERROR: Dataset file not found!")
    raise
```

2.3 Machine Learning Model Implementation

2.3.1 Random Forest Configuration

PYTHON CODE

```
model = RandomForestClassifier(
    n_estimators=100,      # Number of trees
    random_state=42,       # Reproducibility
    max_depth=10,          # Depth limit
    min_samples_split=5    # Split requirement
)
```



Feature Importance Visualization Example

2.3.3 Model Evaluation

Accuracy > 99%: Indicates excellent performance but may suggest overfitting. Use cross-validation to verify. Features with importance < 0.01 may be candidates for removal.

2.4 Interactive Dashboard Development

The dashboard is implemented as a Python class **CareerRecommendationDashboard**, handling initialization, categorization, and prediction logic.

PYTHON CODE

```
class CareerRecommendationDashboard:
    def __init__(self, model, features, courses_list):
        self.model = model
        self.features = features
        # ... logic to build UI ...
```

2.5 Visualization Implementation

The system creates a 2x2 grid dashboard including:

1. Feature Importance (Top 15)
2. Career Distribution (Popular paths)

3. User Selection Coverage
4. Model Performance (Confusion Matrix)

2.6 Deployment and Distribution

To deploy, ensure the notebook and stud.csv are in the same directory. For production-like web deployment, **Streamlit** is recommended as a standalone alternative.

END OF PROJECT MANUAL

Produced for Gwani Abdullahi Jere